

SHDFL: A Secure Hierarchy Decentralized Federated Learning Refining Convergence With Label-Balanced Clusters

Anonymous Submission

No Institute Given

Abstract. <summarize problems and results, 14 to 16 lines>.

Keywords: decentralized federated learning, hierarchy framework, secure aggregation, non-iid, balanced cluster distribution

1 Introduction

Federated learning (FL) promises collaborative model training without centralizing raw data, yet production deployments still struggle with privacy, scalability, and robustness. Classical cross-device FL relies on hub-and-spoke coordinators that ingest plaintext updates, producing single points of failure and exposing contributors to curious servers. Emerging decentralized designs remove the central coordinator but inherit new challenges: clusters must be balanced to mitigate non-IID drift, secure aggregation must tolerate dropouts, and higher-level coordination must propagate consensus models without overwhelming bandwidth or stalling local optimization. Real deployments also demand auditable storage, fault isolation across administrative domains, and observability hooks that explain aggregation delays.

This work presents *SHDFL*, a Secure Hierarchical Decentralized Federated Learning stack that combines the Bonawitz et al. [1] secure aggregation protocol with a configurable hierarchy spanning clusters, states, and nations. Each cluster executes the Secure Aggregation Procedure (SAP) so only quorum-protected averages leave the cluster, while bridge nodes gossip External Cluster Models (ECMs) that feed the timer-driven hierarchy. The hierarchy uses asynchronous pull-based dissemination, letting nodes continue local training while periodically fetching scope checkpoints anchored on IPFS and blockchain. Fan-out redundancy, deterministic rosters, and restart-aware scheduling make the system resilient to transient dropouts, and Round Amplification ensures privacy is preserved even when rounds abort.

To support realistic non-IID data, the topology generator partitions datasets via Dirichlet sampling, computes per-node label histograms, and constructs D-cliques with iterative swapping to balance class distributions. The same control plane integrates Hyperledger anchoring, IPFS storage, and Prometheus/Grafana observability, enabling us to trace every round from local SGD through multi-level aggregation. Beyond functionality, we profile the operational envelope through

transient dropout campaigns, no-SAP ablations, and privacy analyses that validate quorum enforcement.

Our main contributions are:

- **SHDFL architecture.** We design and implement a secure hierarchical decentralized FL system that unifies cluster-level SAP, bridge-mediated ECM gossip, and multi-scope aggregation suitable for both centralized and decentralized settings.
- **Balanced cluster construction.** We propose a topology pipeline that partitions datasets via Dirichlet sampling and employs label-aware D-clique balancing to reduce drift and communication overhead.
- **Scalable hierarchy.** We present a timer-driven hierarchy with configurable policies (replace/interpolate), fan-out quotas, and asynchronous dissemination so that real deployments can scale across administrative regions without blocking local optimization.
- **Secure aggregation integration.** We adapt the Bonawitz Secure Aggregation to cluster rounds with quorum enforcement, restart semantics, and Round Amplification analysis, demonstrating enhanced privacy for SHDFL participants.
- **Empirical evaluation.** We report experiments, privacy analyses, and ablation studies—including dropout scenarios and no-SAP runs—that quantify the trade-offs between accuracy, bandwidth, and privacy in our system.

The rest of the paper is organized as follows. Section 2 introduces related work. Section 3 proposes the architecture and workflow. Section 4 evaluates its effectiveness via experiments. Finally, Section 5 concludes the paper.

2 Related Work

The integration of blockchain with decentralized federated learning (DFL) has gained significant attention as a mechanism to enhance trust, auditability, and coordination in distributed learning systems. Formery et al. [2] provide a comprehensive architectural analysis of Blockchain-Based Decentralized Federated Learning (BDFL), decomposing such systems into layered components including blockchain infrastructure, client interconnection, consensus management, and data transmission. Their work highlights decentralization, immutability, and transparency as key properties that enable trustworthy coordination in open and partially trusted environments. By anchoring model commitments and learning events on-chain, BDFL architectures improve traceability and tamper resistance.

However, these frameworks primarily focus on architectural characterization rather than optimization-aware learning design. The learning layer is typically treated independently from the blockchain layer, with limited consideration of communication topology, data heterogeneity, or aggregation structure. Moreover, most BDFL designs assume a flat peer-to-peer network without structured hierarchy. Our work extends this coordination paradigm by tightly integrating blockchain-based auditability with topology-aware decentralized optimization and multi-level aggregation.

A fundamental challenge in decentralized stochastic gradient descent (D-SGD) is convergence under heterogeneous (non-IID) data distributions. The D-Cliques approach [3] addresses this issue by organizing nodes into densely connected cliques whose aggregated label distributions approximate the global distribution, while maintaining sparse inter-clique connectivity to reduce communication cost. This topology mitigates optimization bias induced by heterogeneous neighborhoods and improves convergence stability.

Le Bars et al. [4] further refine the theoretical understanding of D-SGD by introducing the notion of *neighborhood heterogeneity*. Their analysis demonstrates that convergence depends not only on global heterogeneity but also on the mismatch between a node’s data distribution and that of its neighbors. By explicitly coupling topology and heterogeneity, they show that appropriate topology learning can significantly reduce convergence degradation. These works provide strong theoretical and practical motivation for topology-aware decentralized training.

Nevertheless, both D-Cliques and refined topology learning concentrate on improving convergence within a single-level network and do not address system-level concerns such as verifiable coordination, privacy-preserving aggregation across administrative domains, or hierarchical scalability. In contrast, our framework extends topology-aware principles into a structured hierarchy composed of clusters, states, and a nation-level aggregation layer. At the lowest level, nodes are grouped into statistically balanced clusters inspired by D-Cliques. Multiple clusters form a state, and multiple states contribute to nation-level aggregation. This hierarchy enables scalable coordination across organizational boundaries while preserving topology-aware convergence benefits.

Our topology also allowed us to address privacy, which remains critical in decentralized learning. Bonawitz et al. [1] propose a practical secure aggregation protocol that allows an aggregator to compute only the sum of client updates using pairwise masking and secret sharing, ensuring robustness to dropouts while preventing exposure of individual updates. Although highly effective, this protocol is typically applied in centralized federated learning with a trusted server.

Our framework adapts secure aggregation to the cluster level of a decentralized hierarchy. Within each cluster, a designated cluster head coordinates secure aggregation without accessing individual model updates. At higher levels, aggregation operates on already aggregated cluster models, reducing privacy exposure while enabling scalable learning. This layered aggregation strategy preserves confidentiality while supporting large-scale deployment.

Scalability challenges in blockchain-based federated learning have also motivated off-chain storage solutions. Zhao et al. [5] propose leveraging the InterPlanetary File System (IPFS) to store model parameters off-chain, while anchoring content identifiers (CIDs) and hashes on the blockchain. This separation between verification and storage improves scalability and cost efficiency while maintaining integrity guarantees.

However, existing blockchain-FL systems often adopt flat orchestration models and lack structured multi-level aggregation. Our framework incorporates the blockchain/IPFS separation paradigm within a three-level hierarchy: cluster

models are securely aggregated and stored in IPFS with blockchain anchoring, state-level merging combines multiple clusters, and nation-level aggregation integrates state models. This yields end-to-end traceability and tamper-resistant model lineage across all levels.

Prior work typically advances one dimension at a time—blockchain coordination [2], topology-aware convergence [3,4], secure aggregation [1], or off-chain storage [5]—and leaves the others untouched. Our contribution is to unify these components in a cross-silo setting with stable identities and administrative autonomy. The cluster–state–nation architecture addresses convergence, privacy, auditability, and scalability simultaneously by coupling topology-aware decentralized learning with secure aggregation and blockchain-anchored coordination across all layers, moving blockchain-based decentralized FL closer to practical large-scale deployment.

3 Proposed Method

3.1 System Architecture

Figure 1 sketches the execution stack that ties together the trusted third party (TTP), the cluster-level secure aggregation runtime, the hierarchy-aware overlays, and the storage back-ends. The stack is layered so that data provisioning, protocol orchestration, hierarchical coordination, and anchoring services evolve independently while preserving a coherent privacy boundary.

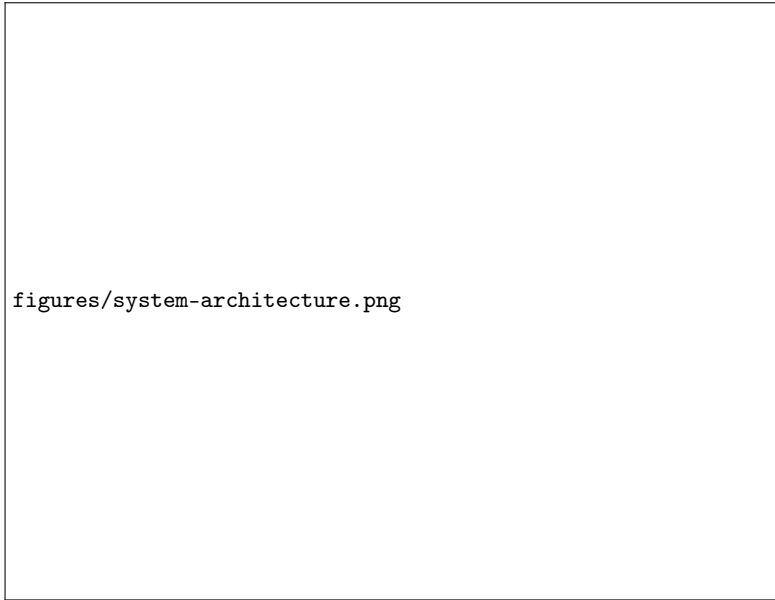


Figure 1: Privacy-preserving federated learning stack with clusters, bridge links, hierarchy timers, and storage back-ends.

Label-Balanced cluster Topology and Membership

The TTP plans topology before training begins. After partitioning the dataset with a Dirichlet sampler, it computes per-node label histograms and feeds them to a topology builder. The builder repeatedly swaps members across candidate clusters to minimize label skew, producing roughly balanced clusters that represent the global distribution. Inter-cluster edges are then laid out using a configurable mode (ring, ring with extra edges, small-world, or similar), yielding bridge nodes that later disseminate ECMs. Thresholds for each cluster are set to $\lceil \frac{2}{3}|C| \rceil$ so the Secure Aggregation Procedure tolerates a minority of failures without sacrificing privacy. The TTP finally distributes three artifacts to every node: its cluster roster, its bridge neighbors, and its canonical identifier for storage anchors. Because the TTP distributes final rosters and identifiers up front, every node already knows exactly which peers share its cluster; after a restart they simply reload that deterministic membership and resume, so the cluster composition never changes dynamically during training and there is no ambiguity about who participates.

Secure Aggregation Procedure in Cluster Rounds

Each cluster-level training round executes the Secure Aggregation Procedure (SAP) so that the elected aggregator learns only the average of the members' model deltas while every contributor's raw tensor remains unknown. Without SAP, trainers would have to upload their plaintext updates to the aggregator, enabling a curious or compromised aggregator to infer label distributions or memorized samples from individual gradients. Embedding SAP directly into the cluster orchestration preserves the privacy boundary proved in the Bonawitz et al. [1] protocol while keeping the rest of the federated pipeline unchanged.

SAP requires a quorum at every round boundary because the server can only reconstruct the pairwise masks and self-masks after it gathers at least t distinct shares for each secret. In practice, the aggregator waits for messages from the surviving clients in each of the four rounds and aborts if the threshold is not met, mirroring the “*abort if $|u_i| < t$* ” guard described in the paper. The quorum therefore guarantees that (i) no coalition smaller than t can strip another client's mask and (ii) the unmasking phase has sufficient information to produce a valid mean; without it, the round is deliberately canceled to avoid revealing partial information or emitting an incorrect aggregate.

When a cluster round aborts due to insufficient survivors, all nodes reset that cluster round pipeline: they resume local SGD from the checkpoint produced in the failed attempt, re-run message advertisement, and execute SAP again. This restart loop can repeat multiple times, which effectively increases the amount of local computation associated with a single logical cluster round (*Round Amplification*). The system tolerates this amplification because it trades extra local epochs for the safety guarantee that only quorum-protected aggregates ever leave the clique.

Nodes that dropped during SAP but later come back online do not interrupt the in-flight protocol. They simply await the next successfully committed cluster CID from IPFS/ledger, fetch it once the clique finishes, and continue training from that baseline. This keeps the view of participants consistent with the synchronous SAP transcript: only the members that remained alive throughout a run contribute to the aggregate, while late arrivals stay passive until the following round.

A more pathological situation occurs when a subset of cluster members fails permanently. In that case the repeated restarts can stall the entire cluster, because the quorum is never reestablished and no fresh cluster CID is produced even though the remaining nodes keep training locally. During higher-level aggregation (see section Hierarchy Structure), the fan-out processes of the stuck clique advertise whether their most recent cluster CID is stale. If so, they forward the latest locally trained tensors as artifacts so the state or nation aggregators can still incorporate the clique’s progress despite the missing secure aggregate. Those active nodes also continue polling upstream anchors and applying state/nation models to their baselines, ensuring they do not fall behind the rest of the federation even when their clique lacks a new SAP result.

Hierarchy Structure

Above the cluster layer, the system deploys a configurable hierarchy whose levels (cluster, state, nation, etc.) are declared through structured configuration. Each level has a scope name, identifier, timer interval, wait budget, merge policy, and fan-out quota. Nodes also ingest a hierarchy map that lists the roster and children of every scope, enabling them to derive their memberships at runtime. During startup, every node instantiates a per-scope runtime that tracks round queues, round-robin pools for aggregator candidates, and the fan-out members appointed to relay models upward.

High-level aggregation is fully time-driven: each scope owns a wall-clock interval, and when its timer fires the member nodes enqueue a round while briefly pausing local optimization for the configured wait window. If a node is the elected aggregator, it receives the latest model artifacts from its child scopes via bridge fan-out, de-duplicates them per child, fetches the referenced tensors from storage, and aggregate the collected model artifacts. Only one candidate commits the result per round: the merged tensor is published to IPFS, anchored on the ledger under the scope identifier, and the commit event is logged for observability. Peers simply watch the ledger to detect completion and mark the round satisfied without running their own merge.

Dissemination back to the lower layers follows an asynchronous pull model. Once the wait window elapses—or when a fresh anchor arrives early—nodes query the ledger, fetch the referenced tensor from IPFS, verify its hash, and apply the scope’s policy (replace, interpolate with a configured α , etc.) before resuming local updates. Each scope records the last applied CID so redundant anchors are ignored, allowing nodes to adopt the most recent confirmed checkpoint even while the next one is being constructed. Because pauses are brief and higher-

level checkpoints are never pushed synchronously, the hierarchy avoids blocking clique-level secure aggregation yet still guarantees that all members eventually incorporate each level’s consensus model. The result is a scalable, timer-driven architecture where scope membership, fan-out redundancy, and storage verification combine to deliver consistent multi-level coordination without exposing individual model updates.

All experiments discussed in this work instantiate two high-level scopes above the clique layer: a state level that aggregates clusters within an administrative region, and a nation level that aggregates the resulting state checkpoints. This two-tier design mirrors plausible deployment scenarios where national stakeholders need both regional fidelity and nationwide rollups. Nonetheless, the hierarchy remains extensible; additional levels can be declared whenever the topology, policy, and operational constraints of a deployment demand them, allowing the same control plane to span everything from metropolitan overlays to multi-national consortia.

3.2 Workflow

Figure 2 captures the high-level choreography across stakeholders, IPFS, and the blockchain registry, while Figures 3 and 4 provide more detailed flows, expanding each block into concrete interactions.

Overall Execution Flow

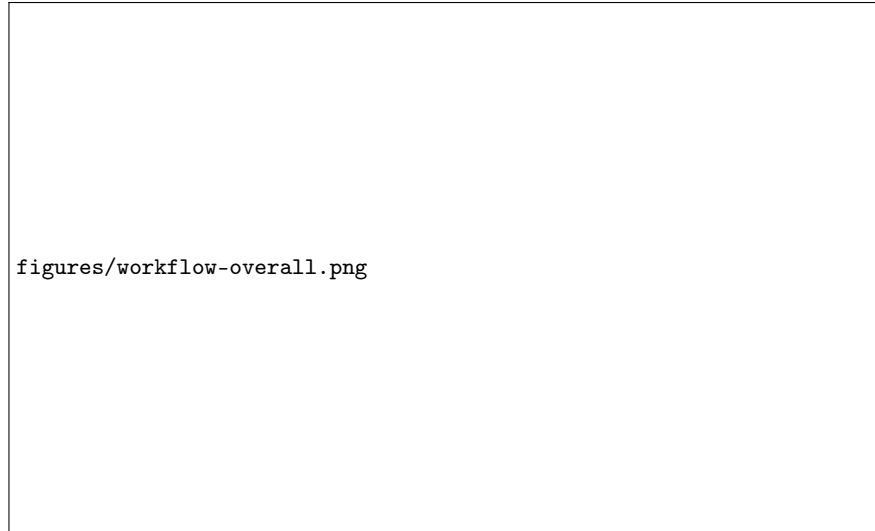


Figure 2: Overall workflow of system.

1. **Configure dataset and training plan.** The system admin selects the dataset split, chooses the reference model, and fixes the per-round training knobs (optimizer, learning-rate schedule, wait windows, and related time budgets). This action ensures that every subsequent component operates on a reproducible specification.
2. **Register trainer whitelist.** Authorized trainers are registered on-chain so that only vetted nodes can join future rounds. The whitelist binds node identities to their scope metadata and enables later audits.
3. **Run D-cliques algorithm at the system tier.** Using the registered roster, the system partitions trainers into balanced cliques, assigns bridge roles, and persists the resulting topology metadata to IPFS/blockchain. This step guarantees unbiased coverage and reproducibility of the overlay.
4. **Train model locally.** Within each round, trainers pull the latest baseline, resume their optimizer states, and consume the local data shard to produce an updated model. This produces the masked inputs required for secure aggregation.
5. **Join cluster round under the secure aggregation procedure.** Trainers exchange advertisements, masked payloads, and unmasking shares with the elected aggregator. The procedure enforces threshold security so that cluster-level averages leak no individual update.
6. **Pull new model.** After the aggregator commits the cluster model to IPFS and anchors the CID/hash on-chain, trainers retrieve the reference to refresh their local baseline before the next local epoch.
7. **Schedule state/nation round.** Independent timers monitor whether the conditions for higher-level aggregation are met. When an interval elapses, the responsible nodes pause local work and prepare to contribute their latest cluster or state artifact.
8. **Participate in state/nation round.** Fan-out nodes dispatch the newest artifacts to the state/nation aggregator, which merges, anchors, and disseminates the resulting model. Other nodes learn of the completion by polling the blockchain and then apply the scope-specific policy (replace/interpolate) before resuming local training.

System Flow in Detail



Figure 3: First part of detail workflow of system.

System bootstrap (Actions 1.1–1.3). The admin first selects the dataset and reference model (1.1), then fixes the full training configuration (1.2) so that nodes later inherit identical hyper-parameters and convergence rules. With those inputs defined, the stakeholder registers the authorized trainers on-chain (1.3), producing an immutable whitelist that records which organizations may contribute gradients during the experiment.

Run D-cliques algorithm (Actions 2.1–2.5). The system elects pre-aggregators (2.1) and fetches the necessary node metadata, including the per-node data distribution (2.2). Using this context, it forms cliques that minimize label skew (2.3) and assigns bridge nodes plus inter-clique edges to guarantee communication reachability (2.4). Finally, it commits the resulting topology artifacts (2.5) to blockchain so participants and auditors reference the same cluster plan.



Figure 4: Second part of detail workflow of system.

Local training & cluster aggregation round (Actions 3.1–3.9). At the start of each round, trainers pull the latest baseline from storage (3.1) and run local optimization on their shards (3.2). They then quantize their updates and submit them through SAP (3.3), enabling the elected aggregator to compute the intra-clique model (3.4). The aggregator subsequently ingests shared artifacts arriving via bridge nodes (3.5), folds the neighbor checkpoints into the cross-clique merge (3.6), and publishes the updated cluster model (3.7). Throughout the round, bridge nodes broadcast CID/hash pairs to adjacent cliques (3.8), while caching inbound models for the upcoming epoch (3.9).

State/Nation rounds (Actions 4.1–4.4). Fan-out nodes deliver the freshest cluster or state models to the higher-level aggregator (4.1), which aggregates the received artifacts (4.2–4.3). Once the new scope model is finalized, the aggregator anchors it on IPFS/blockchain (4.4), enabling every node to detect completion, apply the policy (replace/interpolate), and resume the next training cycle with a consistent global baseline.

3.3 Optimizing SHDFL with state averaging

The state topology relies bridge-based architecture which built on a sparse, small-world topology (ring + power-of-2 hops) to facilitate inter-cluster communication. While Introducing Hierarchy to this further reduces the total number of active communication channels, it creates a propagation bottleneck. Because information must traverse specific “bridge” nodes to move between clusters, and states, the nation model convergence is constrained by the network’s diameter.

To optimize this process, we propose State average, which streamlines information flow by performing full averaging of cluster-level models within each state. Instead of waiting for information to diffuse across bridges, this optimization forces all clusters in state $\mathcal{S}$ to synchronize their representative models:

$$\theta_{\text{clique}_i}^{(k)} \leftarrow \frac{1}{|\mathcal{C}_S|} \sum_{j \in \mathcal{C}_S} \theta_{\text{clique}_j}^{(k-\frac{1}{2})} \quad (1)$$

where $\mathcal{C}_S$ is the set of cliques in the state. By averaging these aggregated clique models which are already reduced representations of the underlying nodes we achieve instant state-wide synchronization. This optimization effectively bypasses the propagation delay of the bridge-based baseline, leading to significantly faster global convergence.

4 Experiments and Analysis

4.1 Environment Settings

Our experiments benchmark convergence across topologies and algorithmic variants to quantify how the proposed decentralized framework mitigates label-skew. We train on CIFAR-10 (50k train, 10k test across 10 classes) using $N = 24$ simulated nodes that exchange messages exactly as deployed.

Data is partitioned via a Dirichlet sampler (small concentration α) so each node receives a highly non-IID shard, forcing the system to solve a challenging consensus problem. Every node trains a lightweight 3-layer ConvNet with SGD (learning rate 0.01, batch size 64), striking a balance between realism and simulation speed.

For each setting we report test accuracy statistics (min/mean/max across nodes) versus training epochs and contrast the results with an idealized fully-connected baseline of equal size. The baseline samples uniformly from the global distribution, removing label skew and serving as an upper bound on achievable performance.

4.2 SHDFL match Convergence Speed of Fully-Connected with a Fraction of the Edges



Figure 5: Comparison of model distribution on the Dirichlet simplex.

4.3 Dropout Scenario Experiment

To measure quorum-failure impact, we ran an experiment where every cluster round randomly drops β trainers (re-sampled each round, possibly within the same clique). If a clique falls below threshold, SAP aborts, cluster round restarts, and prior dropouts rejoin so the clique eventually regains quorum. The experiment purposely excludes permanent failures—we focus on repeated but recoverable SAP aborts rather than irreversibly stuck cliques.



Figure 6: Accuracy comparison with dropout nodes

Despite the repeated aborts, every clique eventually regained quorum and the system completed the 100-round schedule. Accuracy even surpassed the ideal “no-dropout” baseline because of the *Round Amplification* effect : each restart adds extra local SGD before SAP succeeds. The telemetry chart captures this through the `retry epochs` counter, so a round with k aborts contributes k additional epochs beyond the nominal plan. This boost therefore reflects extra computation rather than a fundamental improvement, underscoring that quorum enforcement preserves privacy at the cost of more local work whenever failures occur.

4.4 No-SAP Ablation

We also ran an ablation where the cluster layer disables SAP so trainers send plain-text updates to the aggregator while every other component remains un-

changed. Accuracy after 100 rounds matches the SAP-enabled baseline, confirming that SAP does not change the learning trajectory—it purely safeguards privacy. Communication metrics diverge: compared with the SAP-enabled system (and even more so versus a fully connected topology), the total packet *count* is lower because clusters reduce fan-out, yet the total transmitted *data* is higher whenever SAP is active due to the $17\times$ expansion from mask vectors. Only the ablation run achieves both fewer packets and less data than the fully connected baseline. This underscores the trade-off: SAP preserves formal privacy guarantees at the cost of inflating payloads, whereas the ablated variant saves bandwidth but exposes individual updates.

figures/comm/communication_comparison.png

Figure 7: Communication comparison 24 nodes

4.5 Privacy Analysis

To protect the confidentiality of local model updates, SHDFL integrates the Practical Secure Aggregation Protocol proposed by Bonawitz et al. [1] within each cluster aggregation step. In this design, trainers never transmit their raw model updates directly to the cluster aggregator. Instead, each update is cryptographically masked before transmission, ensuring that individual contributions remain hidden during aggregation.

Let $\mathbf{x}_u$ denote the local model update produced by trainer u . Before sending the update, the node applies a set of pairwise random masks derived from shared secrets with other cluster members. The transmitted value therefore becomes

$$\mathbf{y}_u = \mathbf{x}_u + \sum_{v \in U} \mathbf{p}_{u,v} + \mathbf{p}_u \pmod{m}, \quad (2)$$

where $\mathbf{p}_{u,v}$ represents pairwise masks shared between nodes and $\mathbf{p}_u$ is a private mask. Because pairwise masks are generated with opposite signs between participants, they cancel when all masked updates are summed by the aggregator. Consequently, the aggregator only obtains

$$\sum_{u \in U'} \mathbf{x}_u, \quad (3)$$

which is the aggregated model update for the cluster.

To maintain correctness in the presence of node dropouts, SHDFL adopts the secret-sharing mechanism from the original protocol. Mask seeds are distributed using a threshold secret sharing scheme, allowing the remaining participants to reconstruct the necessary information to remove residual masks if some nodes fail during the protocol execution. This prevents incomplete masking from corrupting the aggregation result.

Within SHDFL, secure aggregation is executed locally inside each cluster of the decentralized topology. As a result, the cluster aggregator learns only the aggregated update and cannot infer any individual gradient. Combined with decentralized communication and cluster-based training, this design ensures that sensitive local training information remains confidential while still enabling collaborative model optimization.

5 Summary

Two directions warrant further investigation. First, SAP imposes a sizable bandwidth premium because masked payloads expand well beyond plain-text tensors; we plan to prototype packet-level compression tailored to masked gradients so cluster rounds keep their privacy guarantees while shrinking the transmissions. Second, the system currently assumes static cluster assignments, so permanent dropouts can strand a cluster at SAP. We therefore intend to design mid-run rebalancing that detects underpopulated cliques and redistributes surviving nodes without breaking the deterministic identifiers, allowing clusters to regain quorum without operator intervention.

References

1. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 1175–1191. ACM (2017). <https://doi.org/10.1145/3133956.3133982>
2. Formery, Y., Mendiboure, L., Villain, J., Deniau, V., Gransart, C.: A framework to design efficient blockchain-based decentralized federated learning architectures. IEEE Open Journal of the Computer Society **5**, 705–723 (2024). <https://doi.org/10.1109/OJCS.2024.3488512>
3. Guijas, P., Valentín, P., et al.: D-cliques: Decentralized federated learning through topology-aware clique formation. In: Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS). IEEE (2023)
4. Le Bars, B., Bellet, A., Tommasi, M., Lavoie, E., Kermarrec, A.M.: Refined convergence and topology learning for decentralized sgd with heterogeneous data. arXiv preprint arXiv:2204.04452 (2022). <https://doi.org/10.48550/arXiv.2204.04452>
5. Zhao, Y., Zhao, J., Wang, J., Chen, X., Zhang, K., Li, B.: Privacy-preserving blockchain-based federated learning for iot devices. IEEE Internet of Things Journal **8**(3), 1817–1829 (2021). <https://doi.org/10.1109/JIOT.2020.3012275>